

Reporte de la práctica

Control de velocidad de un motorreductor en 3 niveles por comandos de voz

1. Datos generales

Materia	Sistemas Programables
Institución	Instituto Tecnológico de Mazatlán (TecNM)
Alumno	Jesús Antonio Rodríguez Peraza — L231000105
Práctica	Práctica 1 — Control de velocidad de un motorreductor en 3 niveles por comandos de voz
Placa utilizada	Arduino UNO R4 WiFi
Driver de potencia	Puente H L298N (módulo con disipador)
Fuente del motor	Batería LiPo 3S, 11.1 V
App móvil	MIT App Inventor (proyecto ControlMotor) ejecutada con MIT AI2 Companion en Android
Componentes	Arduino UNO R4 WiFi, L298N, motorreductor de CD con llanta, batería LiPo 3S, cables dupont macho-hembra, teléfono Android, red Wi-Fi de 2.4 GHz
Repositorio	github.com/Jesus-Jazisenot/Universidad — Sistemas-Programables/Practica-1-Motor-3-Velocidades-por-Voz

2. Objetivo

Variar la velocidad de un motorreductor de CD en tres niveles mediante una señal PWM aplicada al pin de habilitación (ENA) de un puente H L298N, recibiendo la orden desde una aplicación Android por Wi-Fi. La app reconoce comandos de voz en español ("aumentar", "retroceder", "alto") y envía al Arduino el nivel absoluto que debe tener el motor ($v_{el/0}$ a $v_{el/3}$).

3. Diagrama del circuito (conexiones)

L298N	Conecta a	Nota
ENA	Arduino pin 9 (PWM ~)	Entrada de velocidad. Quitar el jumper de ENA
IN1	Arduino pin 8	Sentido de giro
IN2	Arduino pin 7	Sentido de giro
GND	Arduino GND	Tierra común, obligatoria
+12V	Positivo de la batería LiPo	Fuente de potencia del motor
GND	Negativo de la batería	Mismo nodo que el GND del Arduino
OUT1 / OUT2	Terminales del motorreductor	—

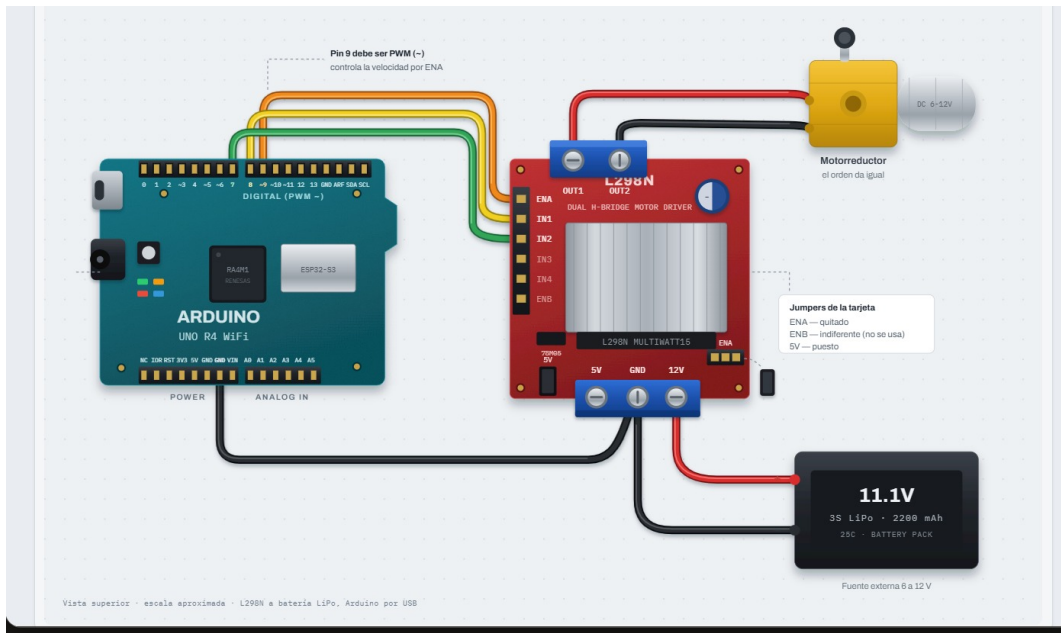


Figura 1. Conexión Arduino UNO R4 WiFi / L298N / motorreductor / batería.

4. Procedimiento

- 1 Se cableó el circuito con todo desconectado, uniendo el GND del Arduino con el GND de la batería y retirando el jumper de ENA del L298N.
- 2 Se cargó `ControlMotor.ino` en el UNO R4 WiFi con el SSID y la contraseña de una red de 2.4 GHz (hotspot del celular).
- 3 En el Monitor Serie (9600 baudios) se leyó la IP asignada y se copió al bloque `initialize global ip` de la app, con `conectado = true`.
- 4 Desde la app se dieron los comandos de voz. Cada comando calcula el nuevo nivel (con tope en 3 y piso en 0) y hace una petición HTTP GET a `/vel/N`.
- 5 El Arduino aplica el PWM del nivel, imprime `Nivel N PWM X` en el Monitor Serie y responde el nombre de la velocidad en texto plano, que la app lee en voz alta.
- 6 Se registró el comportamiento del motor en cada nivel (tabla de la sección 5) y se grabó el video del funcionamiento.

5. Tabla de los tres niveles: PWM aplicado contra velocidad observada

Nivel	Ruta HTTP	PWM	Ciclo de trabajo	Voltaje medio en el motor*	Velocidad observada	Respuesta del Arduino
0	/vel/0	0	0 %	0 V	Detenido. IN1 e IN2 en LOW, el motor queda sin excitación.	detenido
1	/vel/1	130	51 %	≈ 5.7 V	Giro lento y continuo. Arranca directo con la batería cargada; con la batería baja solo zumbaba.	velocidad baja
2	/vel/2	190	75 %	≈ 8.3 V	Giro medio, claramente más rápido que el nivel 1.	velocidad media
3	/vel/3	255	100 %	≈ 11.1 V	Máxima velocidad del motorreductor.	velocidad maxima

* Voltaje medio teórico: ciclo de trabajo × 11.1 V nominales de la LiPo 3S, sin considerar la caída interna del L298N (≈ 1.5–2 V). El motor recibe pulsos a todo el voltaje de la batería, no un voltaje reducido; la velocidad observada es cualitativa, no se contó con tacómetro.

6. Salida del Monitor Serie

Transcripción de lo que imprime el Arduino a 9600 baudios en una secuencia aumentar → aumentar → aumentar → retroceder → alto:

```
Conectando a <red>
....
Conectado
>>> COPIA ESTA IP A LA APP:  http://192.168.x.x
Recibi: GET /vel/1 HTTP/1.1
Nivel 1  PWM 130
Recibi: GET /vel/2 HTTP/1.1
Nivel 2  PWM 190
Recibi: GET /vel/3 HTTP/1.1
Nivel 3  PWM 255
Recibi: GET /vel/2 HTTP/1.1
Nivel 2  PWM 190
Recibi: GET /vel/0 HTTP/1.1
Nivel 0  PWM 0
```

7. Observaciones sobre el comportamiento del sistema

- **El motor no arrancaba en el nivel 1 (PWM 130): solo zumbaba.** Se atribuyó primero a la fricción de la caja de engranes y se agregó una patada de arranque (255 durante 150 ms). La causa real era la **batería descargada**: al bajar su voltaje sube su resistencia interna y, con un ciclo de trabajo del 51 %, el par de arranque ya no vence la fricción estática. Con la batería cargada arranca directo con PWM 130. La patada se dejó en el código como margen, pero la solución es cargar la batería.
- **Sin tierra común el motor no responde** en ningún nivel, aunque todo lo demás esté bien: el L298N no tiene referencia para interpretar IN1, IN2 y ENA. No produce ningún error visible.
- **Con el jumper de ENA puesto el motor gira siempre al máximo:** ENA queda fijo a 5 V y el módulo ignora la señal PWM.
- **La app envía el nivel absoluto y no incrementos.** Si una petición se pierde por la red, la siguiente vuelve a dejar el motor en el estado que muestra la pantalla; el celular y el motor no se desincronizan.

- **La fuente del motor debe ser independiente del Arduino.** Alimentar el motor desde la placa provoca reinicios por la caída de tensión al arrancar.
- El reconocimiento de voz de Google a veces devuelve variantes ("aumenta", "retrocede"); usar fragmentos cortos en el contains de la app atrapa más variantes.

8. Evidencia del armado

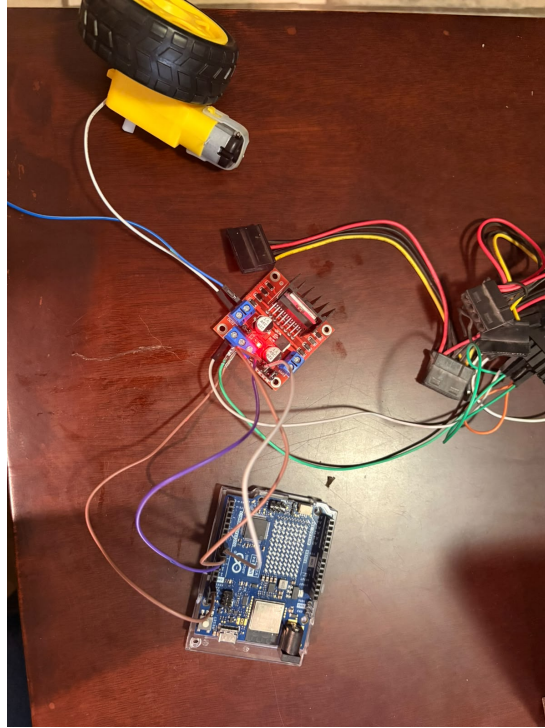


Figura 2. Circuito armado: Arduino UNO R4 WiFi, L298N y motorreductor.

Video del funcionamiento: https://youtu.be/qG1_gLGx5YU